

MS-DOS

The Structure of MS-DOS

- MS-DOS is partitioned into several layers that serve to isolate the kernel logic of the operating system, and the user's perception of the system, from the hardware it is running on.
- These layers are
 - The BIOS (Basic Input/Output System)
 - The DOS kernel
 - The command processor (shell)

The BIOS Module

- **The BIOS is specific to the individual computer system and provided by the manufacturer.**
- **It contains the default resident hardware-dependent drivers for the following devices:**
 - **Console display and keyboard (CON)**
 - **Line printer (PRN)**
 - **Auxiliary device (AUX)**
 - **Date and time (CLOCK\$)**
 - **Boot disk device (block device)**
- **The MS-DOS kernel communicates with these device drivers through I/O request packets; the drivers then translate these requests into the proper commands for the various hardware controllers.**

- In many MS-DOS systems, including the IBM PC, the most primitive parts of the hardware drivers are located in read-only memory (ROM) to be used by stand-alone applications, diagnostics, and the system startup program.
- The BIOS is read into random-access memory (RAM) during system initialisation as part of a file named IO.SYS.
- This file is marked with the special attributes hidden and system.

The DOS Kernel

- The kernel is a proprietary program supplied by Microsoft Corporation and provides a collection of hardware-independent services called system functions.
 - File and record management
 - Memory management
 - Character-device input/output
 - Spawning of other programs
 - Access to the real-time clock

- Programs can access system functions by loading registers with function-specific parameters and transferring them to the operating system using a software interrupt.
- The DOS kernel is read into memory during system initialisation from the MSDOS.SYS file on the boot disk.
- This file is marked with the attributes hidden and system.

The Command Processor

- The command processor, or shell, is the user's interface to the operating system.
- It is responsible for parsing and carrying out user commands, including loading and executing other programs from a disk or other mass-storage device.
- The default shell provided with MS-DOS is found in a file called COMMAND.COM.
- COMMAND.COM can be replaced with a shell of the programmer's own design by simply adding a SHELL directive to the system-configuration file (CONFIG.SYS) on the system startup disk.

More about COMMAND.COM

- The default MS-DOS shell, COMMAND.COM, is divided into three parts:
 - A resident portion
 - An initialisation section
 - A transient module
- The resident portion is loaded in lower memory, above the DOS kernel and its buffers and tables.
- It contains the routines to process Ctrl-C and Ctrl-Break, critical errors, and other transient programs' termination (final exit).
- This part of COMMAND.COM issues error messages and is responsible for the familiar prompt
Abort, Retry, Ignore?

- The resident portion also contains the code required to reload the transient portion of COMMAND.COM when necessary.
- The initialisation section of COMMAND.COM is loaded above the resident portion when the system is started.
- It processes the AUTOEXEC.BAT batch file (the user's list of commands to execute at system startup) if one is present and is then discarded.
- The transient portion of COMMAND.COM is loaded at the high end of memory, and its memory can also be used for other purposes by application programs.

- **The transient module**
 - **issues the user prompt,**
 - reads the commands from the keyboard or batch file and
 - **causes them to be executed.**
- **When an application program terminates, the resident portion of COMMAND.COM checks the transient module to determine whether it has been destroyed and fetches a fresh copy from the disk if necessary.**
- **The user commands that are accepted by COMMAND.COM fall into three categories:**
 - Internal commands
 - **External commands**
 - Batch files

- **Internal commands, sometimes called intrinsic commands, are those carried out by code embedded in COMMAND.COM.**
- **Commands in this category include COPY, REN(AME), DIR(ECTORY), and DEL(ETE).**
- **The routines for the internal commands are included in the transient part of COMMAND.COM.**
- **External commands, sometimes called extrinsic or transient programs, are the names of programs stored in disk files.**

- Before these programs can be executed, they must be loaded from the disk into the transient program area (TPA) of memory.
- Examples of external commands are CHKDSK, BACKUP, and RESTORE.
- As soon as an external command has completed its work, it is discarded from memory; hence, it must be reloaded from disk each time it is invoked.
- Batch files are text files that contain lists of other intrinsic, extrinsic, or batch commands.
- These files are processed by a special interpreter built into the transient portion of COMMAND.COM.

- The interpreter reads the batch file one line at a time and performs each specified operation in order.
- To interpret a user's command, COMMAND.COM first looks to see if the user typed the name of a built-in (intrinsic) command that it can carry out directly.
- If not, it searches for an external command (executable program file) or batch file by the same name.
- The search is carried out first in the current disk drive's current directory and then in each directory specified in the most recent PATH command.

- In each directory inspected, COMMAND.COM first tries to find a file with the extension .COM, then .EXE, and finally .BAT.
- If the search fails for all three file types in all of the possible locations, COMMAND.COM displays the familiar message
Bad command or file name
- If a .COM file or a .EXE file is found, COMMAND.COM uses the MS-DOS EXEC function to load and execute it.
- The EXEC function builds a special data structure called a program segment prefix (PSP) above the resident portion of COMMAND.COM in the transient program area.

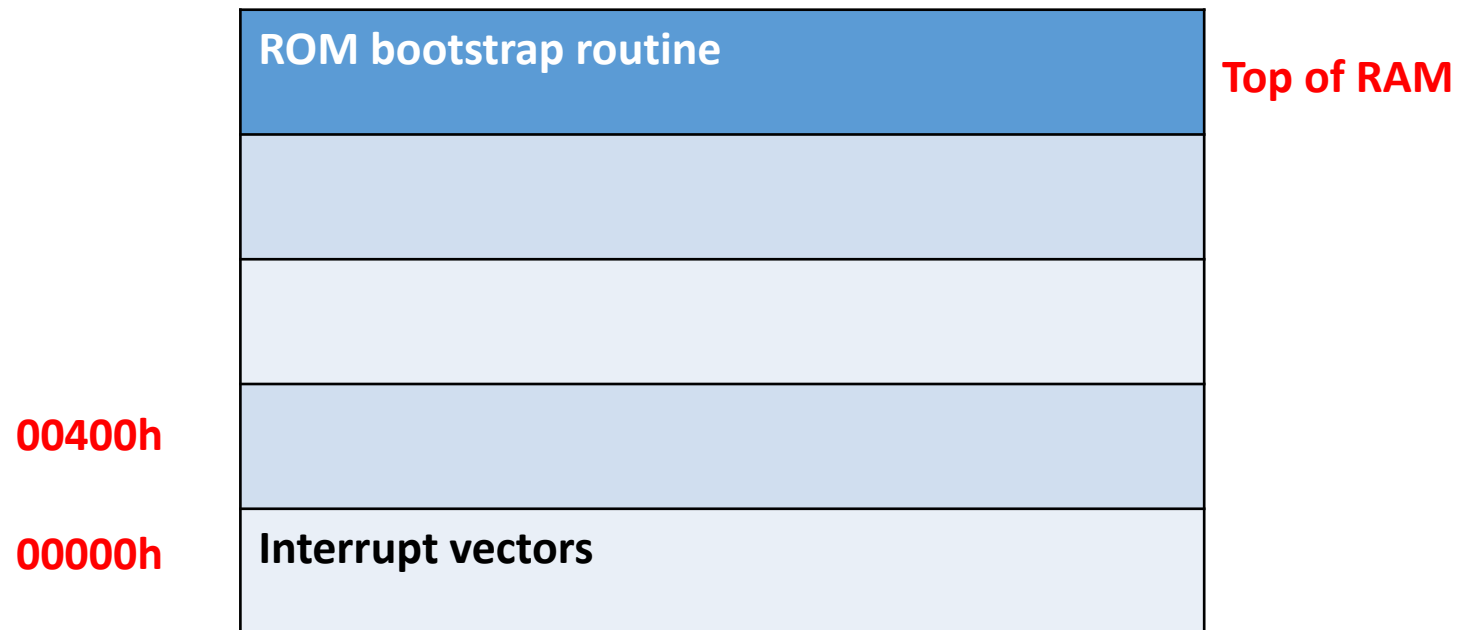
- The PSP contains various linkages and pointers needed by the application program.
- Next, the EXEC function loads the program itself just above the PSP and performs any relocation that may be necessary.
- Finally, it sets up the registers appropriately and transfers control to the entry point for the program.

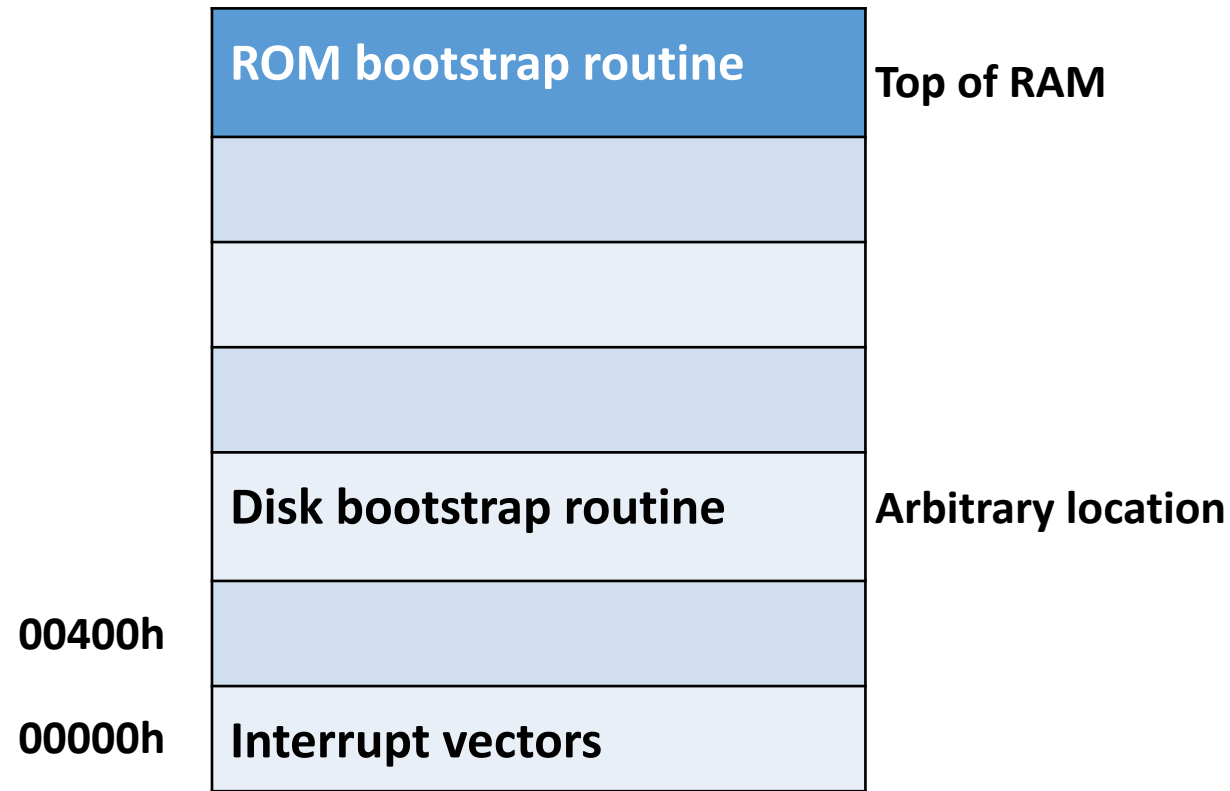
- When the transient program has finished its job, it calls a special MS-DOS termination function that releases the transient program's memory and returns control to the program that caused the transient program to be loaded (COMMAND.COM, in this case).
- A transient program has nearly complete control of the system's resources while executing.
- The only other tasks accomplished are those performed by interrupt handlers (such as the keyboard input driver and the real-time clock) and operations that the transient program requests from the operating system.

How MS-DOS is Loaded

- When the system is started or reset, program execution begins at address 0FFFF0H.
- This is a feature of the 8086/8088 family of microprocessors and has nothing to do with MS-DOS.
- Systems based on these processors are designed so that address 0FFFF0H lies within an area of ROM and contains a jump machine instruction to transfer control to the system test code and the ROM bootstrap routine.
- The ROM bootstrap routine reads the disk bootstrap routine from the first sector of the system startup disk (the boot sector) into memory at some arbitrary address and then transfers control to it.

- (The boot sector also contains a table of information about the disk format.)
- The disk bootstrap routine checks to see if the disk contains a copy of MS-DOS.
- It does this by reading the first sector of the root directory and determining whether the first two files are IO.SYS and MSDOS.SYS in that order.
- If these files are not present, the user is prompted to change disks and strike any key to try again.





The ROM bootstrap routine loads the disk bootstrap routine into memory from the first sector of the system startup disk and then transfers control to it.

- If the two system files are found, the disk bootstrap reads them into memory and transfers control to the initial entry point of IO.SYS.
- The IO.SYS file that is loaded from the disk consists of two separate modules.
- The first is the BIOS, which contains the linked set of resident device drivers for the console, auxiliary port, printer, block, and clock devices, plus some hardware-specific initialisation code run only at system startup.

- The second module, SYSINIT, is supplied by Microsoft and linked to the IO.SYS file, along with the BIOS, by the computer manufacturer.
- **The manufacturer's BIOS initialisation code calls SYSINIT.**
- It determines the amount of contiguous memory in the system and then relocates itself to high memory.

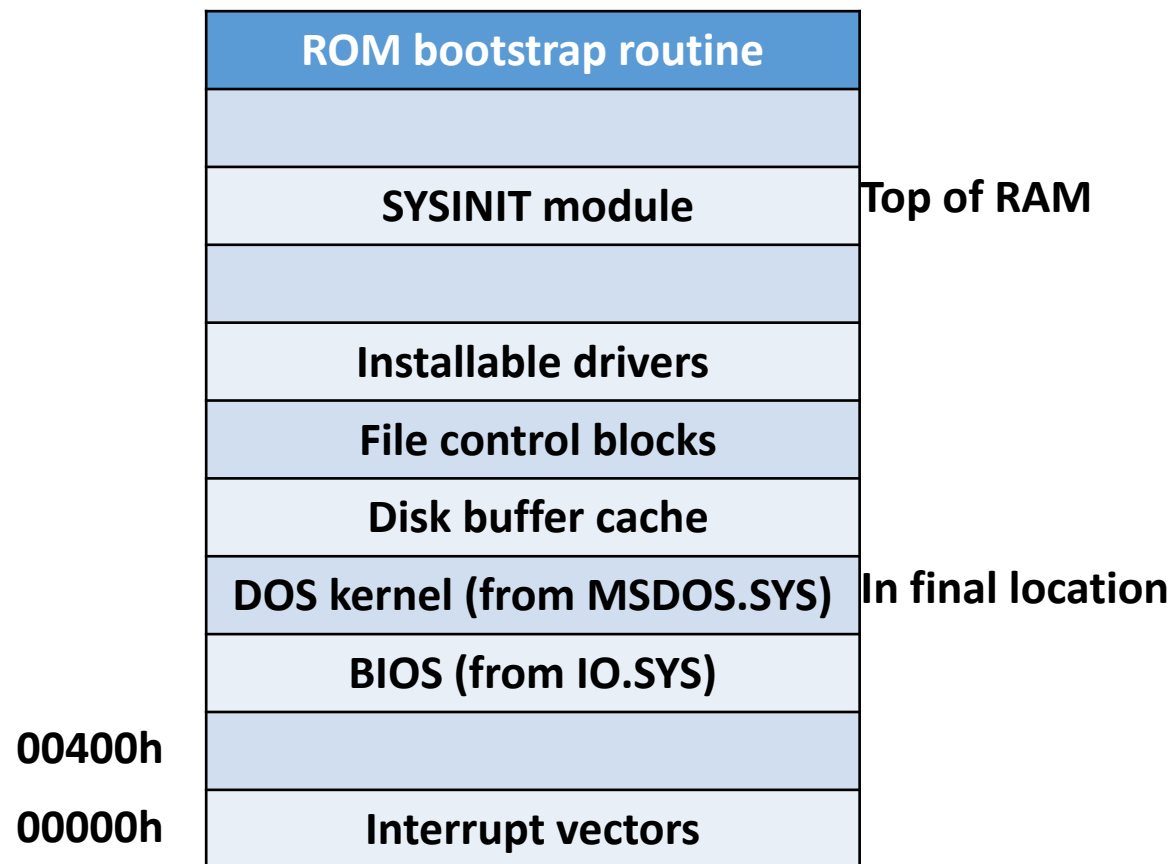
- Then it moves the DOS kernel, MSDOS.SYS, from its original load location to its final memory location, overlaying the original SYSINIT code and any other expendable initialisation code that was contained in the IO.SYS file.
- Next, SYSINIT calls the initialisation code in MSDOS.SYS.
- The DOS kernel initialises its internal tables and work areas, sets up the interrupt vectors 20H through 2FH, and traces through the linked list of resident device drivers, calling the initialisation function for each.

	ROM bootstrap routine	Top of RAM
	Disk bootstrap routine	Arbitrary location
	DOS kernel (from MSDOS.SYS)	Temporary location
	SYSINIT (from IO.SYS)	
	BIOS (from IO.SYS)	
00400h		
00000h	Interrupt vectors	

- These driver functions determine the equipment status, perform any necessary hardware initialisation, and set up the vectors for any external hardware interrupts the drivers will service.
- **As part of the initialisation sequence, the DOS kernel**
 - examines the disk-parameter blocks returned by the resident block-device drivers,
 - **determines the largest sector size that will be used in the system,**
 - builds some drive-parameter blocks, and
 - **allocates a disk sector buffer.**
- Control then returns to SYSINIT.

- **When the DOS kernel has been initialised and all resident device drivers are available, SYSINIT can call the normal MS-DOS file services to open the CONFIG.SYS file.**
- **This optional file can contain a variety of commands that enable the user to customise the MS-DOS environment.**
- **If it is found, the entire CONFIG.SYS file is loaded into memory for processing.**
- **All lowercase characters are converted to uppercase, and the file is interpreted one line at a time to process the commands.**

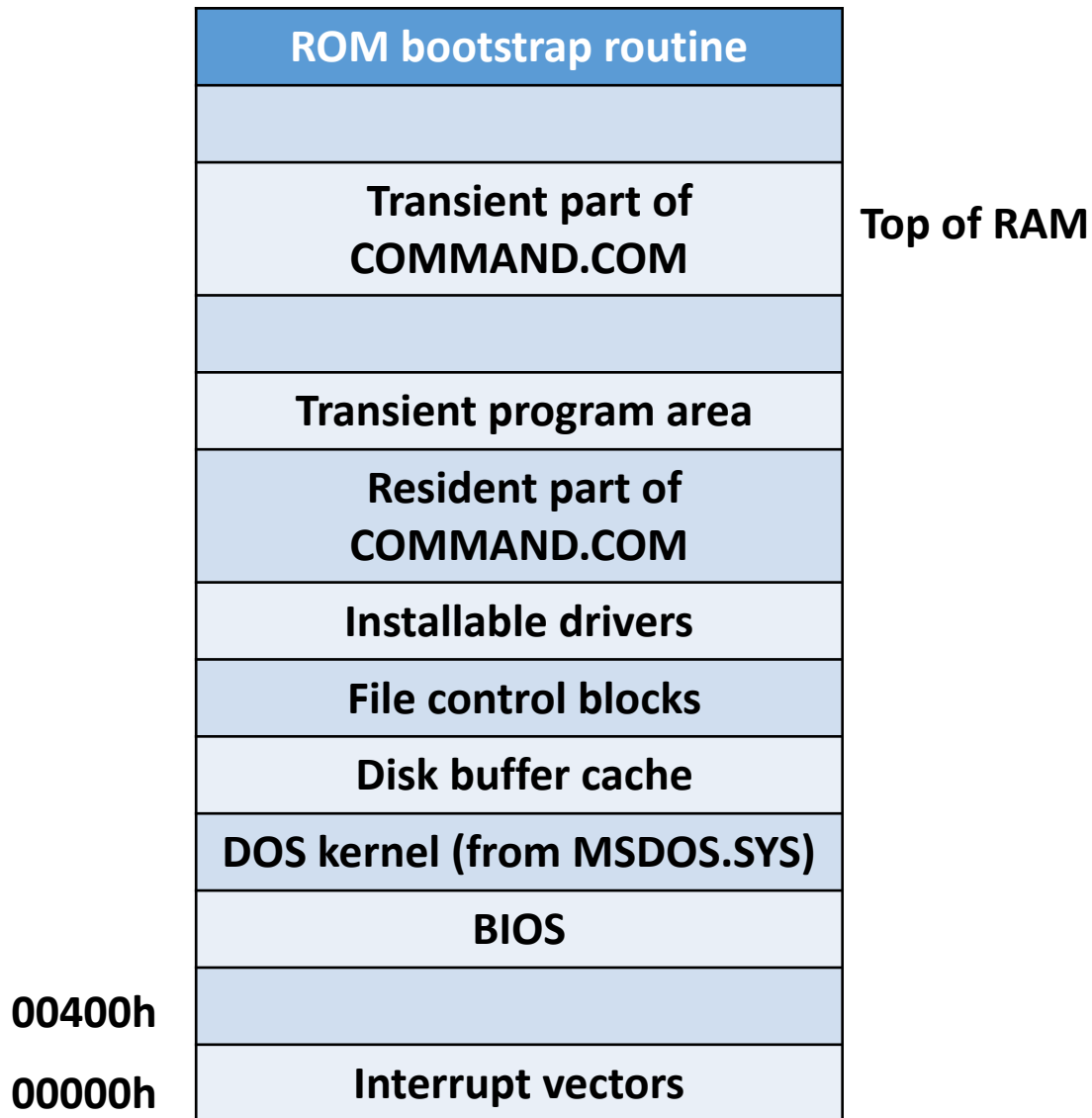
- **Memory is allocated for the disk buffer cache and the internal file control blocks used by the handle file and record system functions.**
- **Any device drivers indicated in the CONFIG.SYS files are sequentially loaded into memory, initialised by calls to their init modules, and linked into the device-driver list.**
- **The init function of each driver tells SYSINIT how much memory to reserve for that driver.**



- **SYSINIT moves itself to high memory and relocates the DOS kernel, MSDOS.SYS, downward to its final address.**

- After all installable device drivers have been loaded, SYSINIT
 - closes all file handles and
 - reopens the
 - console (CON),
 - printer (PRN), and
 - auxiliary (AUX) devices
 - as the standard input, standard output, standard error, standard list, and standard auxiliary devices.
 - This allows a user-installed character-device driver to override the BIOS's resident drivers for the standard devices.

- Finally, SYSINIT calls the MS-DOS EXEC function to load the command interpreter or shell.
- (The default shell is COMMAND.COM, but another shell can be substituted using the CONFIG.SYS file.)
- Once the shell is loaded, it displays a prompt and waits for the user to enter a command.
- MS-DOS is now ready for business, and the SYSINIT module is discarded



- **The resident portion of COMMAND.COM lies in low memory, above the DOS kernel.**
- **The transient portion containing the batch-file interpreter and intrinsic commands is placed in high memory, which can be overlaid by extrinsic commands and application programs running in the transient program area.**

The final result of the MS-DOS startup process for a typical system.

ROM-BIOS

BIOS – Basic Input Output System

- The BIOS is special software that interfaces the major hardware components of your computer with the operating system.
- It is usually stored on a Flash memory chip on the motherboard.
- Its most important role is to load the operating system.
- When a system boots, its BIOS runs the system until the operating system loads and takes control.
- In 2003, Intel introduced UEFI (Unified Extensible Firmware Interface) to completely replace BIOS.

Sequence

- When you turn on your computer, the BIOS does several things. This is its usual sequence:
 1. Check the CMOS Setup for custom settings
 2. Load the interrupt handlers and device drivers
 3. Initialize registers and power management
 4. Perform the power-on self-test (POST)
 5. Display system settings
 6. Determine which devices are bootable
 7. Initiate the bootstrap sequence

Operation: System startup

- **Early intel processors started at physical address 000ffff0h.**
- If the system has just been powered up or the reset button was pressed ("cold boot"), the full power-on self-test(post) is run.
- **If ctrl+alt+delete was pressed ("warm boot"), a special flag value is stored in nonvolatile bios memory (" CMOS") before the processor is reset, and after the reset, the bios startup code detects this flag and does not run the post.**
- This saves the time otherwise used to detect and test all memory.
- The **POST** checks, identifies, and initialises system devices such as the CPU, RAM, interrupt and DMA controllers and other parts of the chipset, video display card, keyboard, hard disk drive, optical disc drive and other basic hardware.

Boot process

- **After POST, the BIOS calls INT 19h to start boot processing.**
- **Post-boot, programs loaded can also call INT 19h to reboot the system. Still, they must be careful to disable interrupts and other asynchronous hardware processes that may interfere with the BIOS rebooting process. Otherwise, the system may hang or crash while it is rebooting.**
- **When INT 19h is called, the BIOS attempts to locate boot loader software held on a storage device designated as a "boot device", such as a hard disk, a floppy disk, a CD, or a DVD.**
- **It loads and executes the first boot software it finds, giving it control of the PC.**
- **This process is known as *booting* which is short for "bootstrapping".**

- The BIOS selects candidate boot devices using information collected by POST and configuration information from EEPROM.
- Following the boot priority sequence, BIOS checks each device to see if it is bootable.
- For a disk drive or a device that logically emulates a disk drive, such as a USB flash drive or perhaps a tape drive, to perform this check, the BIOS attempts to load the first sector (boot sector) from the disk into RAM.
- If the sector cannot be read (due to a missing or unformatted disk or a hardware failure), the BIOS considers the device unbootable and proceeds to check the next device.

- **The BIOS tests each device sequentially until a bootable device is found when the BIOS transfers control to the loaded sector.**
- **Most, but not all, BIOSes load the drive number (as used by INT 13h) of the boot drive into the CPU register before jumping to the first byte of the loaded boot sector.**
- **Note well that the BIOS does not interpret or process the contents of the boot sector other than possibly checking for the boot sector signature in the last two bytes.**
- **All interpretation of data structures like MBR partition tables and so-called BIOS Parameter Blocks is done by the boot program in the boot sector.**

Boot failure

- The original IBM PC and XT had Microsoft Cassette BASIC in ROM, and if no bootable device was found, ROM BASIC was started by calling INT 18h.
- Therefore, barring a hardware failure, an original IBM PC or XT would never fail to boot, either into BASIC or from disk.
- One model of the original IBM PC was available with no disk drive; a cassette recorder could be attached via the cassette port on the rear for loading and saving BASIC programs to tape.
- Since few programs used BASIC in ROM, clone PC makers left it out; then, a computer that failed to boot from a disk would display “No ROM BASIC” and halt (in response to INT 18h).
- Later computers would display a message like “No bootable disk found”; some would prompt for a disk to be inserted and a key to be pressed, and when a key was pressed, they would restart the boot process.
- A modern BIOS may display nothing or automatically enter the BIOS configuration utility when the boot process fails.
- Unlike earlier BIOSes, modern versions are often written assuming that if the computer cannot be booted from a hard disk, the user will not have software they want to boot from removable media instead.

The ROM BIOS Display Functions-INT 10h

Function		Action	Function	Action
Display mode control			Writing to the display	
00H		Set display mode.	09H	Write character and attribute at cursor.
0FH		Get display mode.	0AH	Write character-only at cursor.
Cursor control			0EH	Write character in teletype mode.
01H		Set cursor size	Reading from the display	
02H		Set cursor position.	08H	Read character and attribute at cursor. Graphics support
03H		Get cursor position and size.	0CH	Write pixel.
Scroll or clear display			0DH	Read pixel.
06H		Scroll up or initialize window.	Miscellaneous	
07H		Scroll down or initialize window.	04H	Read light pen.
			05H	Select display page.
			0BH	Select palette/set border color.

Diskette and memory services - INT 13h

Function		Action	Function	Action
00H	Reset Disk System		0EH	Read Sector Buffer
01H	Get Disk System Status		0FH	Write Sector Buffer
02H	Read Sector		10H	Get Drive Status
03H	Write Sector		11H	Recalibrate Drive
04H	Verify Sector		12H	Controller RAM Diagnostic
05H	Format Track		13H	Controller Drive Diagnostic
06H	Format Bad Track		14H	Controller Internal Diagnostic
07H	Format Drive		15H	Get Disk Type
08H	Get Drive Parameters		16H	Get Disk Change Status
09H	Initialize Fixed Disk Characteristics		17H	Set Disk Type
0AH	Read Sector Long		18H	Set Media Type for Format
0BH	Write Sector Long		19H	Park Heads
0CH	Seek		1AH	Format ESDI Drive
0DH	Reset Fixed Disk System			

Serial port – INT 14h

00H	Initialize Communications Port
01H	Write Character to Communications Port
02H	Read Character from Communications Port
03H	Get Communications Port Status
04H	Extended Initialize Communications Port
05H	Extended Communications Port Control

Keyboard Input Methods

- **Applications running under MS-DOS on IBM PC-compatible machines can use several methods to obtain keyboard input:**
 - **MS-DOS handle-oriented functions**
 - **MS-DOS traditional character functions**
 - **IBM ROM BIOS keyboard-driver functions**

MS-DOS handle-oriented functions

- A program uses these functions by supplying a handle, or token, for the desired device, plus the address and length of a buffer.
- When a program begins executing, MS-DOS supplies it with predefined handles for certain commonly used character devices, including the keyboard:

Handle	Device name	Opened to
0	Standard input (stdin)	CON
1	Standard output (stdout)	CON
2	Standard error (stderr)	CON
3	Standard auxiliary (stdaux)	AUX
4	Standard printer (stdprn)	PRN

Keyboard Input with Traditional Calls

- The MS-DOS traditional keyboard functions offer various services with or without echo and Ctrl-C detection.
- The character-input functions all return a character in the AL register.
- For example, the following sequence waits until a key is pressed and then returns it in AL:

```
mov    ah,1      ; function 01h = read keyboard  
int    21h      ; transfer to MS-DOS
```
- The character-input functions differ in whether the input is echoed to the screen and whether they are sensitive to Ctrl-C interrupts.

Int 21H

Function	Action	Ctrl-C checking
01H	Keyboard input with echo	Yes
06H	Direct console I/O	No
07H	Keyboard input without echo	No
08H	Keyboard input without echo	Yes
0AH	Buffered keyboard input	Yes
0BH	Input-status check	Yes
0CH	Input-buffer reset and input	Varies

Keyboard Input with ROM BIOS Functions

- Programmers writing applications for IBM PC compatibles can bypass the MS-DOS keyboard functions and choose from two hardware-dependent techniques for keyboard input.
- The first method is to call the ROM BIOS keyboard driver using Int 16H.
- For example, the following sequence reads a single character from the keyboard input buffer and returns it in the AL register:

```
mov  ah,0      ; function 0=read keyboard  
int  16h      ; transfer to ROM BIOS
```
- Int 16H Function 00H also returns the keyboard scan code in the AH register.
- The other and more hardware-dependent keyboard input method on an IBM PC is to write a new handler for ROM BIOS Int 09H and directly service the keyboard controller's interrupts.

Type	Function
00H	Read Character from Keyboard
01H	Get Keyboard Status
02H	Get Keyboard Flags
03H	Set Repeat Rate
04H	Set Keyclick
05H	Push Character and Scan Code
10H	Read Character from Enhanced Keyboard
11H	Get Enhanced Keyboard Status
12H	Get Enhanced Keyboard Flags

Ctrl-C and Ctrl-Break Handlers

- During most character I/O operations, MS-DOS checks for a Ctrl-C (ASCII code 03H) waiting at the keyboard and executes an Int 23H if one is detected.
- If the system break flag is on, MS-DOS also checks for a Ctrl-C entry during certain other operations (such as file reads and writes).
- Ordinarily, the Int 23H vector points to a routine that terminates the currently active process and returns control to the parent process, usually the MS-DOS command interpreter.
- In other words, if your program is executing and you enter a Ctrl-C, accidentally or intentionally, MS-DOS aborts the program.
- Any files the program has opened using file control blocks will not be closed properly, any interrupt vectors it has altered may not be restored correctly, and if it is performing any direct I/O operations, all kinds of unexpected events may occur.

MS-DOS Display services

- The video display adapters in IBM PCÄ compatible computers have a hybrid interface to the central processor.
- The overall display characteristics, such as vertical and horizontal resolution, background colour, and palette, are controlled by values written to I/O ports whose addresses are hardwired into the adapter.
- In contrast, the appearance of each character or graphics pixel on the display is controlled by a specific location within an area of memory called the regen buffer or refresh buffer.
- Both the CPU and the video controller access this memory; the software updates the display by simply writing character codes or bit patterns directly into the regen buffer. (This is called memory-mapped I/O.)
-

- Under MS-DOS versions 2.0 and later, the preferred method for sending text to the display is to use handle-based Int 21H Function 40H (Write File or Device).
- When an application program receives control, MS-DOS has already assigned its handles for the standard output and standard error devices, which can be used immediately.
- As with the keyboard input functions, MS-DOS also supports traditional display functions.
 - Int 21H Function 02H sends the character in the DL register to the standard output device. It is sensitive to Ctrl-C interrupts.
 - Int 21H Function 06H transfers the character in the DL register to the standard output device, but it is not sensitive to Ctrl-C interrupts.
 - Int 21H Function 09H sends a string to the standard output device. The \$ character terminates the string.

End